The University of Queensland
School of Electrical Engineering and Computer Science

# COMS3200 – Semester 1, 2026
# Assignment (Version 1.1)

Marks: 100
Weighting: 30%
**Due: 3:00pm Friday 8 May, 2026**

Specification changes since version 1.0 are shown in red and are summarised at the end of the document.

## Introduction

The goal of this assignment is for you to design an application layer protocol and demonstrate your knowledge of socket programming, networking and multi-threaded programming.

You have to create client and server applications that implement a publish-subscribe (pub-sub) architecture that allows for clients to publish messages on particular topics and subscribe to messages on particular topics. Your server must deal with multiple connected clients – and allow for "federation" – where a pool of servers supports multiple clients.

You can use C or Python as your coding language (you cannot use any other language). Python 3.11 is the required version if you use Python for this assignment. C code should be compiled with gcc 8.5.0. These are the versions that are available on the student Linux server `moss.labs.eait.uq.edu.au` and that will be made available in the Gradescope marking environment.

You must also document your code using a defined `doxygen` commenting style (described later) that includes details of your use of artificial intelligence (or not). Your assignment mark will be capped at 50% if you do not correctly document all of your source files.

## Student Conduct

**This is an individual assignment**. You should feel free to discuss **general** aspects of C/Python programming and the assignment specification with fellow students, including on the discussion forum. In general, questions like "How should the program behave if ⟨this happens⟩?" would be safe, if they are seeking clarification on the specification.

You must not actively help (or seek help from) other students or other people with the actual design, structure and/or coding of your assignment solution. It is **cheating to look at another person's assignment code or protocol design** and it is **cheating to allow your code or protocol design to be seen or shared in printed or electronic form by others**. All submitted code will be subject to automated checks for plagiarism and collusion. If we detect plagiarism or collusion, formal misconduct actions will be initiated against you, and those you cheated with. If you share your code with another student, even inadvertently, then **both of you are in trouble**. Do not post your code to a public place such as the course discussion forum or a public code repository. Code in private posts to the discussion forum is permitted. You must assume that some students in the course may have very long extensions so do not post your code to any public repository until at least three months after the result release date for the course (or check with the course coordinator if you wish to post it sooner). Do not allow others to access your computer – it is your responsibility to keep your code secure. Never leave your work unattended.

You must follow the following code usage and referencing rules for the code you submit:

| Code Origin | Usage/Referencing |
|---|---|
| **Code provided by teaching staff this semester**<br>Code provided to you **this semester** by COMS3200 teaching staff (e.g., Python and C socket programming examples, posted on the discussion forum by teaching staff, or shown in class). | **Permitted**<br>May be used freely without reference. (You must be able to point to the source if queried about it – so you may find it easier to reference the code.) |

| Code Origin | Usage/Referencing |
|---|---|
| **Unpublished code you wrote earlier**<br>Code you have personally written in a previous enrolment in this course or in another UQ course or for other reasons and where that code has not been shared with any other person or published in any way. | **Conditions apply, references required**<br>May be used (i.e. copied or adapted) provided you understand the code AND the source of the code or learning is referenced in a comment adjacent to that code. For AI use, there must also be an overall comment in each of your source files that uses such code. If such code is used without referencing then this will be considered misconduct. Code references must describe what was used and must specify the source, e.g. including a URL, but there is no specific format to be used. |
| **Code from language/library documentation**<br>Code examples found in `man` pages or official language or library documentation websites. | |
| **Code and learning from AI tools**<br>Code written by, modified by, debugged by, explained by, obtained from, or based on the output of, an artificial intelligence tool or other code generation tool that you alone personally have interacted with, without the assistance of another person. This includes code you wrote yourself but then modified or debugged because of your interaction with such a tool. It also includes code you wrote where you learned about the concepts or library functions etc. because of your interaction with such a tool. It also includes where comments are written by such a tool – comments are part of your code. | |
| **Code copied from sources not mentioned above**<br>Code, in any programming language:<br>• copied from any website or forum (including StackOverflow and CSDN, but excluding official language and/or library documentation websites);<br>• copied from any public or private repositories;<br>• copied from textbooks, publications, videos, apps;<br>• written by or partially written by someone else or written with the assistance of someone else (other than a teaching staff member);<br>• written by an AI tool that you did not personally and solely interact with;<br>• written by you and available to other students; or<br>• from any other source besides those mentioned in earlier table rows above. | **Prohibited**<br>May **not** be used. If the source of the code is referenced adjacent to the code then this will be considered code without academic merit (not misconduct) and will be removed from your assignment prior to marking (which may cause compilation/execution to fail and zero marks to be awarded). Copied code without referencing will be considered misconduct and action will be taken.<br><br>This prohibition includes code written in other programming languages that has been converted to C or Python.<br><br>While you may not copy such code, you may be able to learn from it. See the following row in the table. |
| **Code or resources that you have learned from, other than staff provided code, language/library documentation and AI tool output**<br>Examples, websites, discussions, videos, code (in any programming language), etc. that you have learned from or that you have taken inspiration from or based any part of your code on but have not copied or just converted from another programming language. | **Conditions apply, references required**<br>May be used provided you **do not directly copy code** AND you understand the code AND the source of the inspiration or learning is referenced in a comment adjacent to that code. If such code is used without referencing then this will be considered misconduct. Copying the code itself is prohibited – see the row above this one in this table. There is no specific code referencing format, but the source must be provided, e.g. including a URL. |

**You must not share this assignment specification** or any part of it with any person, organisation, website, etc., with the following exceptions:

- You are permitted to show this specification to course staff and students.
- You are permitted to post extracts of this specification to the course Ed Discussion forum for the purposes of seeking or providing clarification on this specification.
- You are permitted to upload the specification or parts of it to AI and machine translation services but you must be aware of and follow the AI referencing requirements (see the documentation guide), and you should not trust the output of any of these services to be correct.

In short – **Don't risk it!** If you're having trouble, seek help early from a member of the teaching staff. Don't be tempted to copy another student's code or to use an online cheating service (many services have themselves provided evidence of this to the University in the past). Don't help another COMS3200 student

with their code no matter how desperate they may be and no matter how close your relationship. You should read and understand the statements on student misconduct in the course profile and on the school website: `https://eecs.uq.edu.au/current-students/guidelines-and-policies-students/student-conduct`.

# High Level Description

You are to create two programs: `pubsubserver` and `pubsubclient` that implement a publish-subscribe system as described below. Your programs must be built on top of the TCP transport layer. You will need to determine the application layer communication protocol(s) used between the client and the server (and between servers), and what state information is kept in each. Because the protocol is of your creation, we won't be able to test your client and server independently (other than some simple tests such as checking for the handling of erroneous command line arguments and the inability for a client to connect to a server). You will need to implement both a client and a server and have them communicate with each other in order to earn most of the marks for this assignment. We will not be assessing your protocol design directly. We will test your implementation based on the behaviour of your client and server based on inputs supplied to their standard input and by examining what appears on their standard output and/or standard error.

Demo programs will be supplied on `moss`: `demo-pubsubserver` and `demo-pubsubclient`. In all cases, your programs are required to mimic the behaviour of the demo programs (as seen via their `stdin`, `stdout` and `stderr`). You are not expected to mimic the communication protocol used between `demo-pubsubserver` and `demo-pubsubclient` – we have encrypted it so that it doesn't inform your own design.

# Specification – `pubsubclient`

The `pubsubclient` program provides a command line interface for subscribing to particular topics (and therefore receiving messages that match those topics) and for publishing messages on particular topics. The `pubsubclient` can only operate when connected to a `pubsubserver` and will need to be able to simultaneously handle messages from `stdin` and from the server.

## Command Line Arguments

Your `pubsubclient` program is to accept command line arguments as follows (the first version applies if you have used C, the second if you have used Python):

```
./pubsubclient [--topic topic] [server]:port clientid [message]

python3 pubsubclient.py [--topic topic] [server]:port clientid [message]
```

The square brackets (`[]`) indicate optional arguments. The *italics* indicate placeholders for user-supplied value arguments. The option argument (the one that begins with `-`), if present, and its associated value argument must appear before other arguments. Arguments after the optional option argument must be in the order shown.

Some examples of how the program might be run include the following[1]

```
./pubsubclient moss:3200 abc

./pubsubclient --topic temperature :tick-port 123

./pubsubclient --topic rm101/temperature 130.102.72.10:3200 c1 26.0
```

The meaning of the arguments is as follows. More details on the expected behaviour of the program are provided later in this document.

- `--topic` – this option argument specifies that the following value argument (which must be present if this option argument is present) is the initial default topic for messages sent by this client, i.e., the topic that will be used if a message is sent without specifying a topic.
- `[`*`server`*`]:`*`port`* – this argument must be present and optionally specifies the name or IPv4 address of the server to connect to before the colon (`localhost` is to be used if this is not specified) and, after the colon, the port number or service name to connect to. (Service names are taken from `/etc/services`.)

[1] This shows C examples only, is not an exhaustive list and does not show all possible combinations of arguments. The examples assume that a server is listening on the given ports.

- *clientid*– this argument must be present and is a unique identifier for the client. It must be between 2 and 32 characters (inclusive) in length and contain only letters (uppercase or lowercase) and/or digits. (Client identifiers are case sensitive, e.g., "c1" is different to "C1".)
- *message*– this optional argument indicates that `pubsubclient` should immediate publish the given message using the default topic and then exit. It is only valid to specify this argument if the `--topic` argument is present with an associated topic value.

Before doing anything else, your program must check the command line arguments for validity. If the program receives an invalid command line, then it must print the following message:

`Usage: pubsubclient [--topic topic] [server]:port clientid [message]`

to standard error, and exit with an exit status of 1. Note that this message (and all printed messages mentioned below) must be followed by a single newline character.

Invalid command lines include (but may not be limited to) the following:

- the `--topic` option argument is given but is not followed by a value argument (the initial default topic).
- an option argument is listed more than once;
- an unexpected argument is present;
- any argument (or the *port* part of the `[`*server*`]:`*port* argument) is an empty string, i.e. has zero length;
- a *message* argument is provided but a default topic is not specified;
- there is no `[`*server*`]:`*port* argument (i.e. an argument containing a single colon); or
- there is no *clientid* argument.

Option arguments are those beginning with "`--`" prior to the `[`*server*`]:`*port* argument. An argument starting with "`--`" after the `[`*server*`]:`*port* argument is not an option argument – it could be considered a *clientid* or *message* argument depending on its position – and may or may not be valid in that position.

Checking the validity of arguments (e.g., whether a port, client ID or topic is valid) is not part of the usage checking, other than checking that the relevant strings are not empty. Checks on the validity of these arguments are described below and must be carried out in the order given after usage checking is completed successfully.

## Client ID Checking

Client IDs must meet the requirements described above (lines 88 to 90). If not, your client program must print the following message to `stderr`:

`pubsubclient: bad client ID "`*clientid*`"`

where *clientid* is replaced by the client ID argument from the command line. The double quotes must be present[2]. `pubsubclient` must then exit with an exit status of 4.

## Topic Checking

Topics must be a string that starts with a letter (lowercase or uppercase) and consists of letters, numbers, spaces and/or '/' (forward slash) characters. Some sample topics are shown in the following example:

```
humidity
UQ/St Lucia/temperature
River Heights/Brisbane/St Lucia 4067
```

**Example 1**: Example topics

If a valid client ID is provided and the `--topic` argument is given on the command line, then the associated *topic* argument must be checked for validity. If it is not valid, then your program must print the message:

`pubsubclient: invalid topic string "`*topic*`"`

to standard error where *topic* is replaced by the topic argument from the command line. Your program must then exit with an exit status of 5.

[2]Double quote characters shown below in output messages must always be present.

## Message Checking

If a *message* argument is provided then it must contain only printable characters, i.e. characters for which the C function `isprint()` returns true or for which the Python String method `isprintable()` returns True. If non-printable characters are detected then your program must print the message:

```
pubsubclient: messages must only contain printable characters
```

to standard errror and exit with an exit status of 6.

## Connection Checking

If the applicable checks above are successful, then the `pubsubclient` must attempt to connect to the `pubsubserver` specified by the `[`*server*`]:`*port* argument. If this fails (e.g. because the *server* (if specified) and/or *port* is invalid or because there is no process listening on that *port*), then your program must print the message:

```
pubsubclient: unable to connect to "server:port"
```

to standard error where *server*`:`*port* is replaced by the `[`*server*`]:`*port* argument from the command line (with `localhost` inserted if the *server* part is omitted). Your program must then exit with an exit status of 7.

## Server Validity Checking

If the applicable checks above are successful, then the `pubsubclient` must check it has connected to a compatible `pubsubserver`, i.e., one written by you. If not, then, within one second, the client must print the following message to `stderr` and exit with an exit status of 8:

```
pubsubclient: server at "server:port" is not a valid server
```

where *server*`:`*port* is replaced by the `[`*server*`]:`*port* argument from the command line (with `localhost` inserted if the *server* part is omitted).

## Client Uniqueness Checking

If the applicable checks above are successful, then it must be checked whether the client has a unique client ID, i.e., the ID is not used by any other client connected to this server. If the client ID is not unique for this server, then the client must print the following message to `stderr` and exit with an exit status of 9:

```
pubsubclient: client ID "clientid" is not unique
```

where *clientid* is replaced by the client ID from the command line.

## Client Runtime Behaviour

If the usage, client ID, topic, message, connection, server validity and client uniqueness checks (as applicable) are successful, then your `pubsubclient` program must behave as described below.

If a *message* is given on the command line then the client publishes this to the server (with the default *topic*) and should then exit with exit status 0. Nothing should be output to `stdout` or `stderr`.

If a *message* is not given on the command line then the client enters interactive mode. It will first print (and flush[3]) the following message to `stdout`:

```
Welcome to pubsubclient!
```

The client program must then repeatedly (and potentially simultaneously):

- read lines from `stdin` and process them as described below, and
- receive messages from the server and process them as described below.

### Client – Handling Standard Input

Lines[4] read from `stdin` that start with a single forward slash ('`/`', possibly after leading whitespace characters) are commands (potentially with arguments) that are handled as described below. The use of these commands may or may not be associated with communication to or from the server – that is up to you to determine based on the descriptions below, your protocol design and decisions you make about where state information

[3] All messages printed by `pubsubclient` must be flushed.

[4] A line is a sequence of non-null characters of any length terminated by a newline character or EOF. Terminating newline characters are not considered part of the line.

is stored (client and/or server). Note that commands and arguments are separated by one or more whitespace characters[5] and that leading and trailing whitespace characters on a line are not part of any command or argument. Arguments may be enclosed in double quotes if desired – and this is required if an argument contains one or more whitespace characters[6]. The quotes are not considered part of the argument and should be removed before the value is used for anything. If an invalid command is entered, then `pubsubclient` must print the following to `stderr`:

```
pubsubclient: unknown command
```

If a valid command is entered with missing, empty, or extra arguments, or if double quotes are used incorrectly in the argument(s) to a valid command, then `pubsubclient` must print a usage error message to `stderr`:

`pubsubclient: unknown argument(s) - usage:` *`command-usage`*

where *`command-usage`* is replaced by the usage string next to the bullet points below. For example, if `pubsubclient` reads a line like this from `stdin`:

```
/subscribe
```

(which has insufficient arguments), then `pubsubclient` must print the following usage message:

```
pubsubclient: unknown argument(s) - usage: /subscribe topic [filter]
```

If arguments are invalid, then a specific error message will be printed as described below and no other action is carried out. Valid commands are:

- `/subscribe` *`topic`* `[`*`filter`*`]`
  This command indicates that the client wishes to subscribe to the given *`topic`*, optionally with the given *`filter`*. The given topic must meet the requirements set out in the Topic Checking section above. If the *`topic`* is invalid, then `pubsubclient` must print the following to `stderr`:

  `pubsubclient: invalid topic string "`*`topic`*`"`

  where *`topic`* is replaced by the value the user provided.

  The *`filter`*, if present, must take the form:
  *`operator value`*
  where *`operator`* is one of `< <= > >= == !=`, and *`value`* is a numerical value (integer or floating point – but you can treat them as floating point). If there are whitespaces between the *`operator`* and *`value`* then the *`filter`* argument must be enclosed in double quotes.

  If the filter is invalid, then `pubsubclient` must print the following to `stderr`:

  `pubsubclient: invalid filter string "`*`filter`*`"`

  where *`filter`* is replaced by the value the user provided.

  If the `/subscribe` command is valid, then all messages in the system that match the given *`topic`* (and the given *`filter`* value if specified) must be sent from the `pubsubserver` to the `pubsubclient` and reported by the client on its `stdout` (see details later – on lines 286 to 294). If a filter is specified, the message will only be considered a match if (a) the message is a numerical value (integer or floating point, possibly with surrounding whitespace) and (b) the value of the number in the message meets the condition in the filter.

  If the subscription request is identical to a current subscription (i.e., both *`topic`* and *`filter`* (or absence of filter) match), then the subscription request is to be ignored and `pubsubclient` should print the following to `stderr`:

  ```
  pubsubclient: identical subscription ignored
  ```

  A filter is considered to match if the *`operator`* is the same and the *`value`* is the same when considered as a double precision floating point number (e.g., 1 is the same as 1.000 which is the same as 1e0).

- `/unsubscribe` *`topic`*
  This command indicates that the client wishes to remove any and all subscriptions to the given *`topic`* (independent of the presence of any filters on those subscriptions). If the topic is invalid, an error message is printed to `stderr` – the same message as for an invalid topic in the `/subscribe` command.

[5] In this specification, a *whitespace character* is any character, except the newline character, that the C function `issspace()` returns true for (or the Python function `str.isspace()` returns True for if the string was made up of the single character).

[6] Note that an opening double quote may only appear after a whitespace character and a closing double quote may only appear at the end of a line or before a whitespace character. It is not valid for a command or argument itself to contain a double quote character. It is also invalid if there is an odd number of double quote characters in a line.

If the topic is valid, and the client has one or more subscriptions matching that topic, then `pubsubclient` must print the following to `stdout`:

```
pubsubclient: unsubscribed from messages about "topic"
```

where *topic* is replaced by the value the user provided.

If the topic is valid but the client does not have any matching subscriptions, then `pubsubclient` must print the following to `stderr`:

```
pubsubclient: not subscribed to messages about "topic"
```

where *topic* is replaced by the value the user provided.

- `/topic` *topic*
  This command indicates that the given *topic* is to be the default topic to be used for future messages sent by this client (until such time as this command is used again to change the default topic). If the topic is invalid, an error message is printed to `stderr` – the same message as for an invalid topic in the `/subscribe` command.
- `/sendfile` *filename* [*topic*]
  The file with the given name is to be opened and its contents sent as a message using the given topic or the current default topic if a topic is not specified as an argument. This is the only way to send messages that contain non-printable characters. If the file is not able to opened for reading, then `pubsubclient` must print the following message to `stderr`:

  ```
  pubsubclient: unable to open file "filename"
  ```

  where *filename* is replaced by the filename from from the command. If the file can be opened and a topic is given but is invalid, an error message is printed to `stderr` – the same message as for an invalid topic in the `/subscribe` command. If the file can be opened but no topic is specified and there is no default topic set, then `pubsubclient` must print the following message to `stderr`:

  ```
  pubsubclient: no default topic set
  ```

- `/listsubs`
  List (to `stdout`) the current subscriptions for this client (if any) – one per line. Subscriptions will be listed in the order in which `/subscribe` commands were issued and will be printed using the same format as `/subscribe` commands – with no leading or trailing spaces and single spaces separating `/subscribe` and the *topic*, and, if applicable, the *topic* and the *filter*. (The *filter* string printed will the same as the *filter* argument used in the original `/subscribe` command.) Double quotes must be used around the *topic* and/or *filter* if and only if whitespaces are present in that field. If the client has no subscriptions, then the following message must be printed to `stdout`:

  ```
  No subscriptions
  ```

- `/listlimits`
  List (to `stdout`) the current rate limits for this client (if any) – one per line. Limits will be listed in the order in which `/limit` commands were issued to servers in the federation and will be printed using the same format as used for `/limit` commands to the server (see line 486) – with no leading or trailing spaces and single spaces separating `/limit` and the *clientid*, the *clientid* and the *topic*, and the *topic* and the limit value itself. Double quotes must be used around the *topic* if and only if whitespaces are present in that field. If the client is not rate limited on any topics (including if limits have been removed), then the following message must be printed to `stdout`:

  ```
  No limits
  ```

- `/publish` *topic message*
  This command indicates that the given *message* is to be published using the given *topic*. If the topic is invalid, an error message is printed to `stderr` – the same message as for an invalid topic in the `/subscribe` command. If the topic is valid but the message is invalid (contains non-printable characters) then an error message is printed to `stderr` – the same message as if an invalid *message* was given on the command line. Otherwise, the *message* is published to the server using the given *topic*.
- `/quit`
  The `pubsubclient` is to exit with exit status 0. Nothing is printed.

Lines read from `stdin` that do not start with a forward slash character (possibly with leading whitespace characters) and contain at least one non-whitespace character are to be treated as messages to be published to the server using the current default topic. Any newline read from `stdin` is not part of the message. All other characters on the line (including whitespace and double quote characters) are to be treated as part of the message.

If no default topic has been set on the command line or using the `/topic` command then `pubsubclient` won't publish the message and is to print the following message to `stderr`:

```
pubsubclient: no default topic set
```

If a default topic has been set but the message contains non-printable characters, then your client is to print an error message to `stderr` (the same error message as if the invalid message was on the command line) and not publish the message. Otherwise the message is published to the server (and sent on to any clients with matching subscriptions).

If a line read from `stdin` is empty or contains only whitespace characters then `pubsubclient` ignores that line.

If EOF is detected on `stdin`, then `pubsubclient` is to exit with exit status 0 – the same as if the `/quit` command was entered.

### Client – Handling Messages From the Server

The client may receive messages from the `pubsubserver` that it is connected to at any time – see the description of the server behaviour below.

The following types of message require action and/or output by the client beyond the actions and output described above.

- If the client receives a published message (i.e. a message that matches a subscription made by the client, other than one sent with `/sendfile`), then the client is to print a message to `stdout` of the following form:

  *topic*`:` *message* `(`*server*`:`*client*`)`

  where *topic* is replaced by the topic to which the message was published, *message* is replaced by the message, *client* is replaced by the id of the client that sent the message, and *server* is replaced by the id of the server that the sending client is connected to. The message will be printed exactly as it was supplied by the publishing client. No double quotes are used, even if the *topic* or *message* contain whitespace characters.

- If the client receives a file (sent with `/sendfile`) where the topic matches a subscription made by the client[7], then the file must be saved into the client's working directory (the directory where its execution started) with the filename *N*`_`*filename* where *N* is replaced with the count of the number of files received since the client started (1 will be used for the first file) and *filename* will be replaced by the basename of the file that was sent (i.e., excluding any directory part). The receiving client must print the following message to `stdout`:

  *topic*`: received file "`*filename*`" from `*server*`:`*client* `(`*N* `bytes)`

  where *topic* is replaced by the topic to which the file was published, *filename* is replaced by the name the file was saved to, *client* is replaced by the id of the client that sent the file, *server* is replaced by the id of the server that the sending client is connected to, and *N* is replaced by the number of bytes in the file. If the file cannot be saved (e.g., due to a permissions error), then the receiving client must instead print the following message to `stderr`:

  `pubsubclient: cannot save file "`*filename*`"`

  where *filename* is replaced by the name of the file that cannot be saved.

- If the client is informed by the server that a rate limit has been placed on it for some topic (see the description of `/limit` later – line 486 onwards), then it must print the following message to `stdout`:

  `pubsubclient: you are rate limited on topic "`*topic*`" to `*N*` seconds between messages`

  where *topic* is replaced by the identified topic and *N* is replaced by the minimum number of seconds between messages, as advised by the server. If a rate limit is removed (by the server specifying $N = 0$ in a `/limit` command) then this message is still printed with *N* replaced by 0.

[7] Note that there will never be such a match for a "filter" subscription.

If the client tries to publish a message that would exceed the rate limit (i.e. fewer than *N* seconds since the last message on the given topic since this rate limit was set), then the client must print the following message to `stderr`:

```
pubsubclient: message publication failed due to rate limit
```

- If the client is informed by the server that the server is exiting, then it must print the following message to `stdout`:

  ```
  pubsubclient: exiting due to server shutdown
  ```

  and exit with exit status 0.
- If the connection to the server is closed without the server having advised of an orderly shutdown, then the client must print the following message to `stderr` and exit with status 10.

  ```
  pubsubclient: server disconnected - exiting
  ```

### Client – Other Requirements

When a client exits, the server(s) will not retain any subscriptions from the client. This means that if a client connects using the same *`clientid`* then they will have to issue subscription requests again.

It is valid for a client to subscribe to messages that it will publish itself. Matching messages must be reported in the usual way.

## Specification – `pubsubserver`

The `pubsubserver` program manages subscriptions and the publication of messages for the clients connected to it. It is also to support *federation* – where a number of peer servers work together to provide service to a group of clients. If servers are connected together in a federation then all of their clients are treated in the same way as if they were connected to one server, i.e., any message published by a client to any server should be reported to any client connected to any server that has a matching subscription. `pubsubserver` is also to provide a management interface via `stdin`, `stdout` and `stderr`, allowing for clients to be rate limited on particular topics, peer servers to be added, etc.

Each `pubsubserver` will need to be able to simultaneously support communication with multiple clients, other connected servers, and also be able to handle messages from `stdin`.

### Command Line Arguments

Your `pubsubserver` program is to accept command line arguments as follows (the first version applies if you have used C, the second if you have used Python):

```
./pubsubserver [--server [server]:port]... [--listenon listenport] serverid
```

```
python3 pubsubserver.py [--server [server]:port]... [--listenon listenport] serverid
```

The square brackets (`[]`) indicate optional arguments. The *italics* indicate placeholders for user-supplied value arguments. The ellipsis (...) indicates the preceding argument or argument group can be repeated any number of times. Option arguments (those that begin with `--`) and their associated value arguments must appear before other arguments. Option arguments can appear in any order.

Some examples of how the program might be run include the following[8]

```
./pubsubserver --server moss:3200 s1
```

```
./pubsubserver --server moss:3200 --server :3201 jb3200
```

```
./pubsubserver --listenon 3500 --server moss.labs.eait.uq.edu.au:3501 xyz123
```

The meaning of the arguments is as follows. More details on the expected behaviour of the program are provided later in this document.

- `--server` – this option argument (which may appear multiple times) specifies that the following value argument of the form `[`*`server`*`]:`*`port`* specifies the name or IPv4 address of a peer server to connect to

[8] This shows C examples only, is not an exhaustive list and does not show all possible combinations of arguments. The examples assume that the port numbers are being listened on (for `--server` values) or are available to be listened on (for the `--listenon` value).

before the colon (`localhost` is to be used if this is not specified) and, after the colon, the port number or service name to connect to. (Service names are taken from `/etc/services`.)

- `--listenon` – this option argument specifies that the following value argument (*`listenport`*) is the port number or service name on which this server should listen for incoming connections from clients and/or peer servers. (Service names are taken from `/etc/services`.) If this argument pair is not specified then `pubsubserver` will choose a free port to listen on.
- *`serverid`*– this required argument is the ID of the server. It has the same form as a client ID – see lines 88 to 90.

Before doing anything else, your program must check the command line arguments for validity. If the program receives an invalid command line, then it must print the following message:

```
Usage: pubsubserver [--server [server]:port]... [--listenon port] serverid
```

to standard error, and exit with an exit status of 1.

Invalid command lines include (but are not limited to) the following:

- an option argument is given but is not followed by a value argument;
- an option argument is listed more than once (other than `--server`);
- an unexpected argument is present;
- the *`serverid`* argument is missing; or
- any argument (or the *`port`* part of a `[`*`server`*`]:`*`port`* argument) is an empty string, i.e., has zero length.

Checking the validity of arguments (e.g., whether a server, port or server ID is valid) is not part of the usage checking, other than checking that the relevant strings are not empty. Checks on the validity of these arguments are described below and take place after the usage checking.

## Server ID Checking

Server IDs must meet the requirements described above for client IDs (see lines 88 to 90). If not, your server program must print the following message to `stderr`:

```
pubsubserver: bad server ID "serverid"
```

where *`serverid`* is replaced by the server ID argument from the command line. `pubsubserver` must then exit with an exit status of 2.

Note that server IDs and client IDs are in separate namespaces, i.e., it is permissible for a client and a server to have the same ID, but it is not permissible for two clients on the same server to have the same ID nor for two servers in a federation to have the same ID.

## Server Listening

If the applicable checks above are successful, then the `pubsubserver` must attempt to listen for incoming client and peer server connections. If the `--listenon` option argument is given on the command line then the server must attempt to listen on the given *`listenport`* (port or service name) for all IPv4 addresses of the host (including the `localhost` address). If it is unable to do so (e.g., because the port/service is invalid or already being used) then it must print the following message to `stderr` and exit with status 3:

```
pubsubserver: can't listen on port "port"
```

where *`port`* is replaced by the *`listenport`* argument from the command line.

If the `--listenon` option argument is not specified on the command line or port "0" is given as an argument to `--listenon`, then `pubsubserver` must bind to a free port for all IPv4 addresses of the host.

In either case, `pubsubserver` must print and flush[9] the following message to `stderr`:

```
pubsubserver: listening on port port
```

where *`port`* is replaced by the numerical port being listened on.

[9] All messages printed by `pubsubserver` must be flushed.

## Connecting to Peer Servers

If the applicable checks above are successful, then the `pubsubserver` must attempt to connect to any peer servers specified on the command line using the `--server` argument. For each such argument, in the order specified on the command line, the `pubsubserver` server must attempt to:

1. Connect to the peer `pubsubserver` server on the given host (or address) and port number (or service name). If it is unable to connect to that host/port (e.g., because one or both is/are invalid or there is no process awaiting connections on that port) then `pubsubserver` must print the following message to `stderr`:

   `pubsubserver: can't connect to peer "`*`argvalue`*`"`

   where *`argvalue`* is replaced by the `[`*`server`*`]:`*`port`* value as it appeared on the command line.

2. If it is able to connect to the given host/port then it must validate that it is communicating with a compatible `pubsubserver` (i.e., one written by you). If it is unable to do so (within no more than one second), then it must disconnect and print the following message to `stderr`:

   `pubsubserver: Peer server not found at "`*`argvalue`*`"`

   where *`argvalue`* is replaced by the `[`*`server`*`]:`*`port`* value as it appeared on the command line.

3. If the peer server is itself (i.e. a server tries to connect to itself), then it must disconnect and print the following message to `stderr`:

   `pubsubserver: Can't connect to self as peer`

4. If the peer server is a compatible `pubsubserver` but this server is already directly connected to it (possibly using a different IP address), then it must disconnect and print the following message to `stderr`:

   `pubsubserver: Already connected to peer server at "`*`argvalue`*`"`

   where *`argvalue`* is replaced by the `[`*`server`*`]:`*`port`* value as it appeared on the command line. (This check can be based on whether a server with the same ID is directly connected.)

5. If the peer server is a compatible `pubsubserver`, then a check must be made if this federation will result in duplicate server IDs within the federation. If duplicate server IDs will result across the federation then `pubsubserver` must disconnect and print the following message to `stderr`:

   `pubsubserver: Unable to connect to server "`*`argvalue`*`" due to common server IDs`

   where *`argvalue`* is replaced by the `[`*`server`*`]:`*`port`* value as it appeared on the command line. (Note that it is permissible for the peer server to already be part of this server's federation – not via a direct connection but via one or more other peers. This does not count as a duplicate server ID. A duplicate server ID would be another server connecting with the same ID.)

6. If the checks above (as applicable) pass and the server has connected successfully to the peer server, then the server must print the following to `stdout`:

   `pubsubserver: Connected to peer "`*`serverid`*`" at "`*`argvalue`*`"`

   where *`serverid`* is replaced by the server ID of the server connected to and *`argvalue`* is replaced by the `[`*`server`*`]:`*`port`* value as it appeared on the command line. The server being connected to, must print the following message to its `stdout`:

   `pubsubserver: Connection received from peer "`*`serverid`*`"`

   where *`serverid`* is replaced by this server's server ID.

Note that `pubsubserver` is not to exit if any of these connections fail. It is permissible for a server to have multiple links to servers within the same federation, but it is not permitted for a server to have multiple links to the same server.

## Server Runtime Behaviour

Once the server has completed the checks above, then it must behave as described below.

It must repeatedly (and potentially simultaneously):

- read lines from `stdin` and process them as described below, and
- receive messages from clients and peer servers and process them as described below.

### Server – Handling Standard Input

Lines read from `stdin` that start with a single forward slash ('`/`', possibly after leading whitespace characters) are commands (potentially with arguments) that are handled as described below. The use of these commands may or may not be associated with communication to or from clients or peer servers – that is up to you to determine based on the descriptions below, your protocol design and decisions you make about where state information is stored (client and/or server). Note that commands and arguments are separated by one or more whitespace characters and that leading and trailing whitespace characters on a line are not part of any command or argument. Arguments may be enclosed in double quotes if desired – and this is required if an argument contains one or whitespace characters[10]. The quotes are not considered part of the argument and should be removed before the value is used for anything. If an invalid command or a line that does not start with a single forward slash is entered, then `pubsubserver` must print the following to `stderr`:

```
pubsubserver: unknown command
```

If a valid command is entered with missing, empty or extra arguments, then `pubsubserver` must print a usage error message to `stderr`:

```
pubsubserver: unknown argument(s) - usage: command-usage
```

where *`command-usage`* is replaced by the usage string next to the bullet points below. If arguments are invalid, then a specific error message will be printed as described below.

Valid commands are:

- `/listclients [--all]`
  This command will list the IDs of clients connected to this server in the form *`serverid:clientid`*. If the `--all` option argument is given then the IDs of clients connected to all servers in the federation will be listed (with the id of the server they are connected to). (If there are no peer servers then the list will be the same as if the `--all` argument was not provided.) The lines are always listed in ASCII sort order and are printed one per line to the server's `stdout`. If no clients are connected then the server is to print the following to `stdout`:

  ```
  pubsubserver: No clients connected
  ```

- `/listpeers [--all]`
  This command will list the IDs of servers connected directly to this server. This includes servers to which this server initiated the connection and also servers that initiated the connection to this server. If the `--all` option argument is given then the IDs of all peer servers in the federation will be listed. Each ID will be listed exactly once, even if there are multiple paths to the server. Server IDs are always listed in ASCII sort order and are printed one per line to the server's `stdout`. If no servers are connected to this server then the server is to print the following to `stdout`:

  ```
  pubsubserver: No peer servers connected
  ```

- `/peer [`*`server`*`]:`*`port`*
  This command indicates that this server should attempt to connect to a peer server. This is handled in the same way as if this was specified with a `--server` argument on the command line (see the Connecting to Peer Servers section above).

- `/limit` *`clientid topic N`*
  This command indicates that messages from the client (on this server) with the given client ID on the given topic are to be rate limited to no more than one every *`N`* seconds. (This includes the sending of files.) A value of zero indicates that there is no rate limit. If *`clientid`* is not a valid client ID or is not the ID of a client connected to this server then `pubsubserver` must print the following message to `stderr`:

  ```
  pubsubserver: Client "clientid" is unknown
  ```

  where *`clientid`* is replaced by the value the user provided.

  If the client is valid, but the topic is not valid, then `pubsubserver` must print the following message to `stderr`:

  ```
  pubsubserver: Topic "topic" is not valid
  ```

  where *`topic`* is replaced by the value the user provided.

[10] Note that an opening double quote may only appear after a whitespace character and a closing double quote may only appear at the end of a line or before a whitespace character. It is not valid for a command or argument to contain a double quote character. If double quotes are present, then there must be an even number of double quotes on the line.

If the client and topics are valid but the *N* argument is not a non-negative integer between 0 and 3600 inclusive, then `pubsubserver` must print the following message to `stderr`:

```
pubsubserver: Rate limit must be 0 to 3600 seconds inclusive
```

There can only be one rate limit per client per topic so this command will replace a rate limit if one has been previously specified. If a rate limit is set or modified, then past message publications are irrelevant – the limit only applies to future published messages. The client can therefore publish a message on this topic immediately but the message after that will not be sent to subscribers if it comes too soon after the last (within the specified rate limit period).

It is up to you whether rate limiting is implemented client side or server side – the key requirement is that clients with subscriptions that match the message must not be sent the message if the publishing client exceeds its rate limit.

The client must be informed when a limit is put in place on it, even if a limit is removed by specifying *N* as zero. The same applies if an *N* value of zero is used an there is no existing limit in place – the client is informed but no limit is put in place.

- `/quit`
  This command indicates that the server is to exit. It must communicate that it is exiting to connected clients and peer servers. The server must then exit with status 0. Peer servers must handle this gracefully (i.e., tidy up resources as required) and print a message as described below (line 537). Connected clients will exit as described on lines 319 to 322.

If the server detects EOF on its `stdin` then it must behave in the same manner as if the `/quit` command was entered.

### Server – Handling Messages From Clients and Peer Servers

The server may receive messages from a connected `pubsubclient` or a connected peer `pubsubserver` at any time. The following types of message or event require action and/or output by the server beyond the actions and output described above.

- When a client connects, the server must satisfy itself within one second that it is a compatible `pubsubclient` (i.e., one written by you), and if so and the client has a unique ID in the federation, the server must print the following message to its `stdout`:

  ```
  pubsubserver: Client "client" has connected
  ```

  where *client* is replaced by the client's ID.

  If a connection is received from a compatible `pubsubclient` but the client ID would be duplicated, the server must print the following message to its stdout:

  ```
  pubsubserver: Client ID "client" would be duplicated - aborting connection
  ```

  where *client* is replaced by the duplicated ID.

  If a connection is received from some other program, then your server must close that connection within one second and print the following message to its `stderr`:

  ```
  pubsubserver: Connection with unknown client aborted
  ```

- When a client disconnects, the server must print the following message to its `stdout`:

  ```
  pubsubserver: Client "client" has disconnected
  ```

  where *client* is replaced by the client's ID. Any subscriptions made by that client are no longer valid.
- When a peer server advises that it is shutting down, then the server must print the following message to its `stdout`:

  ```
  pubsubserver: Peer server "serverid" shutting down
  ```

  where *serverid* is replaced by the disconnected server's ID.
- When a peer server disconnects without advising this server that it is doing so ~~details of an alternative server to connect to~~, then this server must print the following message to its `stderr`:

  ```
  pubsubserver: Peer server "serverid" disconnected
  ```

  where *serverid* is replaced by the disconnected server's ID.

- When a client publishes a message, any clients in the federation that have a matching subscription must be sent the message. If a client has more than one matching subscription then the message must only be sent once (or at least only printed once by the client).

### Other Requirements

- All messages printed to `stdout` or `stderr` must be followed by a single newline character and must be flushed (unless the program is exiting immediately, in which case the system will flush the message). Double quote characters shown in messages must be present in the output.
- We will not test for unexpected system call or library function failures in an otherwise correctly implemented program (e.g., resource allocation failure). Your program can behave however it likes in these cases, including crashing.
- For any aspects of behaviour not fully described in this specification, your program must behave in the same manner as `demo-pubsubclient` and `demo-pubsubserver` on `moss`. If you're unsure about some aspect, then you can run the demo programs to check the expected behaviour.

## ~~Style~~

~~Your submission must comply with the COMS3200 Code Documentation Requirements available on the course Blackboard site. This outlines strict commenting and referencing requirements applicable to both C and Python code.~~

## Hints

1. Review the sample code provided that relates to network clients, threads and synchronisation (semaphores/locks), and multi-threaded network servers. This assignment builds on all of these concepts.
2. Remember to flush output to `stdout` and network sockets if using high level output primitives (e.g., `fprintf()` in C). Output may not be newline buffered.
3. You will need to use appropriate mutual exclusion in your server to avoid race conditions when accessing common data structures and shared resources.
4. The federation behaviour adds significant complexity. You may wish to complete single-server behaviour first – though keep federation in mind when you design your protocol.

## Testing

You are responsible for ensuring that your program operates according to the specification. You are encouraged to test your program on a variety of scenarios.

A variety of programs will be provided to help you in testing:

- Demonstration programs (called `demo-pubsubclient` and `demo-pubsubserver`) that implement the correct behaviour are available on `moss`. You can test your programs with a given scenario and test the demo programs with a given scenario to check that you get the same behaviour. You can also use the demonstration programs to check the expected behaviour of the programs if some part of this specification is unclear. Note that you cannot test your client with the demo server or vice versa as the protocol used by the demo programs is not being made available to you.
- A test script will be provided on `moss` that will test your programs against a subset of the functionality requirements – approximately 50% of the available marks. The script will be made available about 7 to 10 days before the assignment deadline and can be used to give you some confidence that you're on the right track. The "public tests" in this test script will not test all functionality, and you should be sure to conduct your own tests based on this specification and the demo programs. The "public tests" will be used in marking, along with a set of "private tests" that you will not see until results are released.
- The Gradescope submission site will also be made available about 7 to 10 days before the assignment deadline. Gradescope will run the test suite immediately after you submit. When this is complete[11] you

[11] Gradescope marking may take only a few minutes or more than 30 minutes, depending on the functionality and efficiency of your code.

will be able to see the results of the "public tests". You should check these test results to make sure your program is working as expected. Behaviour differences of the same code between `moss` and Gradescope may be due to memory initialisation assumptions in your code, so you should allow enough time to check (and possibly fix) any issues after submission.

# Submission

You must submit a zip file to Gradescope that contains your code at the top level of the zip file. Only files matching the following names will be extracted and considered in marking:

- `*.c`
- `*.h`
- `*.py`
- `Makefile` (or `makefile`)

Other files and any subfolders in your zip file will be ignored for marking purposes. We will then build/check your code for marking as follows. (Note that ***`path`*** in the commands below is replaced by the directory path to your program – which may or may not be the current directory.)

- If a `Makefile` (or `makefile`) exists then the marking script will run `make` (with no target specified).
- If `make` was run and an executable file named `pubsubclient` was created then we will run your client as

  ***`path`*`/pubsubclient`**
- If a `pubsubclient` executable does not exist but `pubsubclient.py` exists then we will run your client as

  `python3` ***`path`*`/pubsubclient.py`**
- If neither `pubsubclient` nor `pubsubclient.py` exists then we will not test your client and you will receive zero marks for client functionality and any functionality that requires both a server and a client.
- If `make` was run and an executable file named `pubsubserver` was created then we will run your server as

  ***`path`*`/pubsubserver`**
- If a `pubsubserver` executable does not exist but `pubsubserver.py` exists then we will run your server as

  `python3` ***`path`*`/pubsubserver.py`**
- If neither `pubsubserver` nor `pubsubserver.py` exists then we will not test your server and you will receive zero marks for server functionality and any functionality that requires both a server and a client.

Multiple submissions to Gradescope are permitted. We will mark whichever submission you choose to "activate" – which by default will be your last submission, even if that is after the deadline and you made submissions before the deadline. Any submissions after the deadline[12] will incur a late penalty – see the COMS3200 course profile for details.

# Marks

Marks will be awarded for functionality. Marks will be capped at 50% if you do not meet the AI documentation requirements. Marks may be scaled if you attend an interview about your assignment and you are unable to adequately respond to questions – see the Student Interviews section below.

## Functionality (100 marks)

Provided your executables can be checked/generated as above, then you will earn functionality marks based on the number of features your program correctly implements, as outlined below. Not all features are of equal difficulty. Partial marks will be awarded for partially meeting the functionality requirements. A number of tests will be run for each marking category listed below, testing a variety of scenarios. Your mark in each category will be proportional (or approximately proportional) to the number of tests passed in that category. If your program does not allow a feature to be tested then you will receive zero marks for that feature, even if you claim to have implemented it. For example, if your client can never create a connection to a server then we can not determine whether it can publish messages or not. Your tests must run in a reasonable time frame, which

[12] or your extended deadline if you are granted an extension.

could be as short as a few seconds for usage checking to many tens of seconds in some cases. If your program takes too long to respond, then it will be terminated and you will earn no marks for the functionality associated with that test. Exact text matching of output (`stdout` and `stderr`) is used for functionality marking. Strict adherence to the formats in this specification is critical to earn functionality marks. The markers will make no alterations to your code (other than to remove code without academic merit).

Marks will be awarded in the following categories. Note that, other than for the indicated standalone tests, your client and server will be tested together and both the client and server need to be working correctly together to earn marks. Note that some functionality will be tested in multiple categories, e.g., correctly matching published messages to subscriptions is required in many tests.

Standalone tests – `pubsubserver` and `pubsubclient` tested independently (17 marks)

1. `pubsubclient` correctly handles invalid command lines (3 marks)
2. `pubsubclient` correctly handles invalid client IDs, topics and messages on the command line (3 marks)
3. `pubsubclient` correctly handles inability to connect to a server and server validity checking (2 marks)
4. `pubsubserver` correctly handles invalid command lines (3 marks)
5. `pubsubserver` correctly handles invalid server IDs (2 marks)
6. `pubsubserver` correctly handles inability to listen on port (2 marks)
7. `pubsubserver` correctly listens on port (includes handling connections from unknown clients) (2 marks)

Tests with a single `pubsubserver` – no federation (50 marks)

8. Single `pubsubserver` implements client uniqueness checking (2 marks)
9. Single `pubsubserver` allows single `pubsubclient` to subscribe to topics and publish messages (no filtering, no `stdin` to server, includes handling invalid `/subscribe` and `/publish` commands) (5 marks)
10. Single `pubsubserver` allows multiple clients to subscribe to and publish messages (no filtering, no `stdin` to server) (5 marks)
11. Single `pubsubserver` allows subscription and publication with filtering (no `stdin` to server, includes handling invalid filters in `/subscribe` commands) (5 marks)
12. Single `pubsubserver` supports identification of duplicate subscriptions and handles unsubscriptions (no `stdin` to server, includes handling invalid `/unsubscribe` commands) (5 marks)
13. Single `pubsubserver` supports setting and changing of default topic (no `stdin` to server, includes handling invalid `/topic` commands) (4 marks)
14. Clients with single `pubsubserver` support `/listsubs` (with a variety of client connections and subscriptions, includes handling invalid `/listsubs` commands) (5 marks)
15. Single `pubsubserver` supports `/listclients` command and `/listpeers` command (with a variety of client connections, includes handling invalid commands for both) (5 marks)
16. Single `pubsubserver` supports disconnecting/reconnecting clients and terminating server (includes testing of functionality above, includes handling invalid `/quit` commands) (4 marks)
17. Single `pubsubserver` supports rate limiting of clients (excluding sending of files, includes `/listlimits` command on client, includes handling invalid `/limit` and `/listlimits` commands) (5 marks)
18. Single `pubsubserver` supports sending of files (including rate limiting and handling invalid `/sendfile` commands) (5 marks)

Tests with multiple `pubsubservers` – federation (33 marks)

19. `pubsubserver` supports connecting (or not) to peers specified on the command line and `/listpeers` command (no message publishing, includes checks for valid connections such as duplicate client/server IDs) (5 marks)

20. `pubsubserver` supports connecting (or not) to peers via `/peer` command and `/listpeers` command (no message publishing, includes checks for valid connections such as duplicate client/server IDs, includes handling invalid `/peer` commands) (5 marks)
21. System supports `/listclients --all` command across a federation of servers with clients (5 marks)
22. System supports clients publishing and subscribing across a federation of servers (no file sending, all peer connections valid, includes rate limiting) (8 marks)
23. System supports clients sending files across a federation of servers (all peer connections valid) (5 marks)
24. System handles disconnecting/reconnecting clients, servers and peer servers (5 marks)

## AI Documentation Requirements and Mark Capping

Every Python and/or C source file that you submit (i.e. files whose names end in `.py`, `.c`, `.h`) must begin with a Doxygen style comment with the following format:

- The first line of the comment (which must be the first line of the file) must contain an `@file` *`filename`* command where *`filename`* is replaced by the name of the file.
- As many `@ai` commands as apply from the following list must be included – each on a separate line in the initial file comment. No other text must be present on these lines.
  - `@ai Not Used` – indicates that AI was not used in any way in the development of this file. No other `@ai` commands must be present in the file.
  - `@ai Wrote Code` – indicates that an AI tool wrote some or all of the code in this file.
  - `@ai Wrote Comments` – indicates that an AI tool was used to write some or all of the comment text (possibly including doxygen comments) in this file.
  - `@ai Inspiration` – indicates that you took inspiration from an AI tool (e.g., discovered the existence of a library function). (This attribute is assumed and need not be specified if the `@ai Wrote Code` command is present.)
  - `@ai Debugging` – indicates that an AI tool was used to help in the debugging of the code in this file.
  - `@ai Testing` – indicates that an AI tool was used to help test the code or to develop test resources
- If `@ai Not Used` is present then no other comments are necessary.
- If one or more of the other `@ai` commands is present, then there must also be:
  - one or more `@aitool` commands which specify the name of each AI tool used (e.g. ChatGPT, Claude, etc. – specific model versions are not required but can be included if desired).
  - a multi-line `@aidetails` command that specifies how the tool was used, including details of which functions/methods/functionality the tool was used for, etc.

Some examples are shown below:

```c
/** @file example.c
 * @author J Smith
 * @ai Wrote Code
 * @ai Debugging
 * @aitool ChatGPT
 * @aitool Claude
 * @aidetails Claude was used to initially
 *  write all of the functions in this
 *  file. ChatGPT was used to debug the
 *  command line argument parsing ...
 *  (details omitted for example)
 */
```

**Example 2**: Sample C file comment 1

```python
"""! @file example.py
@author J Smith
@ai Wrote Code
@ai Debugging
@aitool ChatGPT
@aitool Claude
@aidetails Claude was used to initially write
    all of the functions in this file. ChatGPT
    was used to debug the command line argument
    parsing ... (details omitted for example)
"""
```

**Example 3**: Sample Python file comment 1

```c
/// @file example2.c
/// @author J Smith
/// @ai Not Used
/// @ai Debugging
/// @brief Other doxygen commands such as
///     @author and @brief can be used as
///     desired
```

**Example 4**: Sample C file comment 2

```python
## @file example2.py
# @author J Smith
# @ai Not Used
# @brief Other doxygen commands such as @author
#    and @brief can be used as desired
```

**Example 5**: Sample Python file comment 2

If any source file is missing the required comment then your functionality mark is capped at 50%. Gradescope will report any mark capping after you make a submission (when the test suite completes running – which may take up to tens of minutes depending on the functionality and performance of your code).

## Total Mark

Let

- $F$ be the functionality mark for your assignment (out of 100);
- $D$ be the AI documentation mark cap (either 50 or 100); and
- $I$ be the scaling factor (0 to 1) determined after interview (see the Student Interviews section below) – or 0 if you fail to attend a scheduled interview without having evidence of exceptional circumstances impacting your ability to attend – or 1 if you not required to attend an interview.

Your total mark for the assignment ($M$) will be:

$$M = min\{\text{F,D}\} \times I$$

out of 100.

### Late Penalties

Late penalties will apply as outlined in the course profile.

## Student Interviews

**The teaching staff will conduct interviews with a subset of COMS3200 students about their assignment submission.**

Interviews will take place from the week or two following your submission. If you are selected for interview, you will be emailed a booking link where you can sign up for an interview slot. Invitations to interview may be issued any time up until the end of week one of the exam period.

Students will be selected for interview based on a number of factors that may include (but are not limited to):

- Feedback from course staff based on observations in class, on the discussion forum, and during marking;
- Use of unusual or uncommon code structure/functions etc.;
- Referencing, or lack of referencing, present in code;
- Use of, or suspicion of undocumented use of, artificial intelligence or other code generation tools; and
- Reports from students or others about student work.

The interview is for the purposes of establishing your understanding of the code you submitted and the protocol you designed. If you write your own code and design your own protocol, you have nothing to fear from this process. If you legitimately use code from other sources (and reference it as required) then you are expected to understand that code. If you are not able to adequately explain the design of your solution (including the application layer protocol(s)) and/or adequately explain your submitted code (including the use and behaviour of library functions and system calls) and/or be able to make or describe simple modifications to your code as requested at the interview, then your assignment mark will be scaled down based on the level of understanding you are able to demonstrate. If it becomes clear that you did not write code that you claim is your own or you have not referenced the source of any of your code correctly then your submission may be subject to a misconduct investigation where your interview responses form part of the evidence.

Interviews will usually take about 10 minutes, but some may be shorter (e.g., if your level of understanding becomes apparent quickly) and some may be longer. **You must bring your own laptop to your interview and be ready to show your submitted code (e.g., via Gradescope or using `vim` on `moss`). You will be required to join a Zoom meeting so your screen and the interview is recorded. You must bring your UQ student card or government ID to your interview so that your identity can be confirmed.**

You may not access any notes or other resources (such as websites, AI tools and man pages) during the interview – you may only look at the code you submitted (which must not be modified after submission). You are encouraged to appropriately comment your code before submission to refresh or aid your understanding when you are interviewed about the code.

Scaling to zero marks is possible if no or very limited understanding is demonstrated.

Failure to attend an interview or failure to respond to an interview invitation will result in zero marks for the assignment unless there are documented exceptional circumstances that prevent you from attending, in which case a single reschedule of the interview will be permitted.

# Specification Updates

Any errors discovered in the assignment specification will be corrected and new versions will be released with adequate time for students to respond before the due date. Potential specification errors or inconsistencies can be discussed on the discussion forum. If your program's expected behaviour is not described in this specification, then you can use the demo programs (`demo-pubsubclient` and `demo-pubsubserver`) to determine the expected behaviour. We will not update this specification when the expected behaviour is missing from this specification, only when this specification is inconsistent with itself or with the behaviour of the demo programs or where it is not reasonably possible to determine the expected behaviour from the demo programs.

## Version 1.1

- Replaced `-` by `--` on multiple lines describing command line arguments and arguments within server commands. (This was a LaTeX formatting issue.) Option arguments always begin with `--` and not just `-`.
- Removed Style section (page 14) – the documentation requirements can be found on page 17
- Clarified that marking category 7 also includes handling connections from unknown clients (line 649)
- Clarified how port number "0" should be treated as a server port to listen on (line 396)
- Clarified which marking categories are associated with handling invalid commands (lines 653 to 680)
- Clarified what happens when a rate limit of zero is used and what the client prints when a rate limit is removed (lines 254, 313 and 508)
- Removed out of date details about alternative servers when a peer server disconnects (line 541)